

Principia

Spherical Colour Map System

Implementation Specification

This document specifies the mathematical foundations and implementation details for the two-artefact spherical colour map explorer developed for the Principia three-body visualiser project. It covers colour science, the von Mises–Fisher blending engine, all pattern modes, the colour-vision deficiency simulation pipeline, and the shape sphere physics overlay.

Version 1.0 • Artefacts 01_colour_maps and 02_patterns_special

Written in conjunction with the Principia three-body fractal renderer.

Contents

1	Foundations	3
1.1	The Encoding Problem	3
1.2	Colour Spaces	3
1.2.1	Linear sRGB	3
1.2.2	OKLAB	3
1.3	Chroma and Hue in OKLAB	4
2	The von Mises–Fisher Blending Engine	4
2.1	Von Mises–Fisher Distribution	4
2.2	6-Pole OKLAB Colour Map	4
2.3	Seamless LUT Sphere	5
2.4	Artefact 1: Colour Map Modes	5
3	Colour-Vision Deficiency Simulation	6
4	Rendering Pipeline	7
4.1	Sphere Projection	7
4.2	Rotation	7
4.3	Equirectangular Projection	7
4.4	Specular Glint	7
5	Pattern Modes (Artefact 2)	7
5.1	Voronoi-Type Patterns	8
5.1.1	Octant (8 regions)	8
5.1.2	Voronoi 6	8
5.1.3	Hemispheres	8
5.1.4	Icosahedral	8
5.1.5	Fibonacci Lattice	8
5.1.6	Soft Voronoi	8
5.2	Stripe Patterns	9
5.2.1	Checkerboard	9
5.2.2	Latitude Stripes	9
5.2.3	Longitude Stripes	9
5.2.4	Truchet Mosaic	9
5.3	Overlay Modes	9
5.3.1	Grid Overlay	9
5.3.2	Iso-Hue Contours	9
5.3.3	Gradient Magnitude	10
5.3.4	3-D Perlin Noise	10
5.4	Special Modes	10
5.4.1	Real Spherical Harmonics	10
5.4.2	Turing-Like Standing Waves	10
5.4.3	Custom N-Pole VMF	10
5.4.4	Stability $\times$ Hue	11
5.4.5	Basin Blend	11
5.4.6	Dot Lattice	11
6	Shape Sphere Physics Overlay	11
6.1	Configuration Space	11
6.2	Gaussian Blob Overlay	12
7	Palette System	12

7.1	Fixed- <i>N</i> Palettes	12
7.2	Variable- <i>N</i> Palettes	12
7.3	Cubehelix Generation	12
8	Implementation Notes	12
8.1	Performance	12
8.2	Seam-Free Guarantee	13
8.3	Colour-Blindness Design Principles	13
9	Quick Reference	13

Foundations

The Encoding Problem

The goal is to assign a colour to every point on the unit sphere $S^2 \subset \mathbb{R}^3$. Each point is a unit normal vector $\hat{\mathbf{n}} = (n_x, n_y, n_z)^T$ with $\|\hat{\mathbf{n}}\| = 1$. We require:

1. **Continuity.** Nearby points on S^2 receive similar colours. No discontinuities or seams.
2. **Perceptual uniformity.** Equal angular distances on S^2 correspond to equal perceived colour differences.
3. **Reserved lightness dimension.** Brightness (L) is reserved for an independent scalar field (e.g. orbital stability, basin depth). The colour map must operate at fixed L .
4. **Full 2-D encoding.** Both degrees of freedom on S^2 (latitude and longitude) must be discriminable from colour alone.

Colour Spaces

Linear sRGB

The display pipeline works in linear sRGB. The gamma expansion and compression functions are:

$$\gamma(v) = \begin{cases} v/12.92 & v \leq 0.04045 \\ \left(\frac{v+0.055}{1.055}\right)^{2.4} & v > 0.04045 \end{cases} \quad (1)$$

$$\Gamma(c) = \begin{cases} 12.92 c & c \leq 0.0031308 \\ 1.055 c^{1/2.4} - 0.055 & c > 0.0031308 \end{cases} \quad (2)$$

where $v \in [0, 1]$ is a gamma-encoded channel value and $c \in [0, 1]$ is a linear light value.

OKLAB

OKLAB (Ottosson, 2020) is a perceptually uniform colour space with axes:

- $L \in [0, 1]$: lightness (0 = black, 1 = white)
- $a \in \mathbb{R}$: green–red axis
- $b \in \mathbb{R}$: blue–yellow axis

The conversion from OKLAB to linear sRGB proceeds via two matrix multiplications and a cube-root non-linearity:

$$\begin{pmatrix} \ell' \\ m' \\ s' \end{pmatrix} = M_1 \begin{pmatrix} L \\ a \\ b \end{pmatrix}, \quad \begin{pmatrix} \ell \\ m \\ s \end{pmatrix} = \begin{pmatrix} \ell'^3 \\ m'^3 \\ s'^3 \end{pmatrix} \quad (3)$$

$$\begin{pmatrix} R_\ell \\ G_\ell \\ B_\ell \end{pmatrix} = M_2 \begin{pmatrix} \ell \\ m \\ s \end{pmatrix} \quad (4)$$

where

$$M_1 = \begin{pmatrix} 1 & 0.3963 & 0.2158 \\ 1 & -0.1056 & -0.0639 \\ 1 & -0.0895 & -1.2915 \end{pmatrix}, \quad M_2 = \begin{pmatrix} 4.0767 & -3.3077 & 0.2310 \\ -1.2684 & 2.6098 & -0.3413 \\ -0.0042 & -0.7034 & 1.7076 \end{pmatrix}$$

The (R_ℓ, G_ℓ, B_ℓ) values are then gamma-compressed by $\Gamma(\cdot)$ and clamped to $[0, 255]$.

Chroma and Hue in OKLAB

It is convenient to parameterise the (a, b) plane in polar form. Define:

$$a = C \cos(\theta_h), \quad b = C \sin(\theta_h) \quad (5)$$

where $C \geq 0$ is the *chroma* and $\theta_h \in [0, 2\pi)$ is the *hue angle*. The (L, C, θ_h) parameterisation is called OKLCH. Throughout this system, hue angles are assigned to sphere poles and chroma is a global parameter controlling vibrancy.

The von Mises–Fisher Blending Engine

Von Mises–Fisher Distribution

The von Mises–Fisher (vMF) distribution on S^2 is the spherical analogue of a Gaussian. Its probability density at $\hat{\mathbf{n}} \in S^2$ given mean direction $\hat{\mathbf{p}} \in S^2$ and concentration $\kappa > 0$ is:

$$f(\hat{\mathbf{n}}; \hat{\mathbf{p}}, \kappa) \propto \exp(\kappa \hat{\mathbf{n}} \cdot \hat{\mathbf{p}}) \quad (6)$$

The density is maximum at $\hat{\mathbf{n}} = \hat{\mathbf{p}}$ and decays smoothly with geodesic distance $d = \arccos(\hat{\mathbf{n}} \cdot \hat{\mathbf{p}})$.

6-Pole OKLAB Colour Map

Place $N = 6$ *colour poles* at the axis intersections:

$$\hat{\mathbf{p}}_1 = \hat{x}, \quad \hat{\mathbf{p}}_2 = -\hat{x}, \quad \hat{\mathbf{p}}_3 = \hat{y}, \quad \hat{\mathbf{p}}_4 = -\hat{y}, \quad \hat{\mathbf{p}}_5 = \hat{z}, \quad \hat{\mathbf{p}}_6 = -\hat{z}$$

Assign each pole a hue angle $\theta_i \in [0, 2\pi)$ such that opposing poles are separated by π (complementary colours). Two pole schemes are implemented:

Pole	Direction	Full OKLAB Colour	θ_i	Okabe–Ito θ_i
$\hat{\mathbf{p}}_1$	$+\hat{x}$	Red	0°	250° (Blue)
$\hat{\mathbf{p}}_2$	$-\hat{x}$	Cyan	180°	70° (Orange)
$\hat{\mathbf{p}}_3$	$+\hat{y}$	Green	120°	30° (Vermillion)
$\hat{\mathbf{p}}_4$	$-\hat{y}$	Magenta	300°	210° (Sky Blue)
$\hat{\mathbf{p}}_5$	$+\hat{z}$	Blue	240°	170° (Bluish Green)
$\hat{\mathbf{p}}_6$	$-\hat{z}$	Yellow	60°	350° (Reddish Purple)

The (a, b) coordinates at a query point $\hat{\mathbf{n}}$ are computed as a vMF-weighted mean in the OKLAB (a, b) plane:

$$a(\hat{\mathbf{n}}) = C \cdot \frac{\sum_{i=1}^N w_i \cos \theta_i}{\sum_{i=1}^N w_i}, \quad b(\hat{\mathbf{n}}) = C \cdot \frac{\sum_{i=1}^N w_i \sin \theta_i}{\sum_{i=1}^N w_i}, \quad w_i = \exp(\kappa \hat{\mathbf{n}} \cdot \hat{\mathbf{p}}_i) \quad (7)$$

Key properties of (7):

- **Seamless.** No `atan2`, no branch cut, no discontinuity anywhere on S^2 .
- **Complementary cancellation.** At the midpoint between opposing poles, $\hat{\mathbf{n}} \cdot \hat{\mathbf{p}}_i = \hat{\mathbf{n}} \cdot \hat{\mathbf{p}}_{i+1}$, so $w_i = w_{i+1}$ and the $\cos \theta_i + \cos \theta_{i+1} = 0$ terms cancel, giving $a = b = 0$ (neutral grey). This is physically correct.

- **Concentration parameter κ .** Controls the spread of each blob. Low κ ($\lesssim 2$): soft, wide blobs. High κ ($\gtrsim 8$): crisp poles with large neutral regions.
- **Perceptual uniformity.** All operations are linear in OKLAB (a, b) , so the weighted mean is a straight-line blend in a perceptually uniform space.

Seamless LUT Sphere

Matplotlib-style 1-D colour LUTs (Viridis, Cividis, Plasma, etc.) define a sequential colour ramp over $t \in [0, 1]$. To apply such a LUT seamlessly to S^2 , we employ the same vMF blending engine:

1. Sample $N_e = 16$ colours from the LUT at $t_i = i/N_e$ for $i = 0, \dots, N_e - 1$ and assign them to equatorial poles at angles $\phi_i = 2\pi i/N_e$:

$$\hat{\mathbf{p}}_i^{\text{eq}} = (\cos \phi_i, \sin \phi_i, 0)$$

2. Add north and south poles with LUT endpoint colours: $\hat{\mathbf{p}}_N = (0, 0, 1)$ at $t = 1$ and $\hat{\mathbf{p}}_S = (0, 0, -1)$ at $t = 0$.
3. Blend in RGB (not OKLAB) using the same vMF weights.

The seam-free property holds because the equatorial poles are placed at uniform angular intervals and the blending is a smooth exponential weighting — there is no angle computation at query time.

Note on Twilight. The Matplotlib Twilight LUT is cyclic ($t = 0$ and $t = 1$ yield the same colour), making it ideal for longitude encoding. The equatorial-poles approach naturally exploits this: adjacent poles at $\phi = 0$ and $\phi \rightarrow 2\pi$ receive the same colour, so the longitude seam is entirely invisible.

Artefact 1: Colour Map Modes

The following colour map modes are implemented in `01_colour_maps.jsx`:

Mode	Method	Notes	CB Safety
VMF OKLAB	6-pole vMF, eq. (7), full-spectrum hues	Perceptually uniform, fully 2-D. L , C , κ adjustable.	Deutan issues
VMF Okabe–Ito	6-pole vMF, Okabe–Ito hues	Same engine; blue–orange axis replaces red–green. All 6 poles distinguishable under all common CVDs.	Safe
Viridis	Seamless LUT sphere, $N_e = 16$, $\kappa = 4$	Perceptually uniform sequential.	Safe
Cividis	Seamless LUT sphere	Optimised so deuteranopes and trichromats perceive identical orderings.	Optimised
Plasma	Seamless LUT sphere	Vivid. Purple–red range problematic for some CVDs.	Partial
Magma	Seamless LUT sphere	Dark sequential; partially greyscale-readable.	Partial
Turbo	Seamless LUT sphere	Maximum vibrancy; full rainbow. Not CB-safe.	Not safe
Inferno	Seamless LUT sphere	Black $\rightarrow$ purple $\rightarrow$ orange $\rightarrow$ yellow.	Partial

Mode	Method	Notes	CB Safety
Twilight	Seamless LUT sphere	Cyclic LUT; longitude seam fully invisible.	Partial
Cool–warm	Seamless LUT sphere	Diverging blue → white → red. Good for bipolar fields.	Partial
Principia	Seamless LUT sphere, custom palette	Deep indigo → midnight blue → teal → warm gold.	Partial
Cubehelix	Seamless LUT sphere, analytical	Dave Green’s formula: hue spirals with monotone lightness increase. CB-tolerant by construction.	Tolerant
Direction cosines	$R = (1 + n_x)/2$, $G = (1 + n_y)/2$, $B = (1 + n_z)/2$	Three independent channels, each encoding one axis. Not perceptually uniform. Collapses in true greyscale.	Better grey

Global controls (Artefact 1):

- **Invert.** All output values mapped $v \mapsto 255 - v$ in sRGB before CVD simulation.
- **Auto-rotate.** Increments the azimuthal rotation angle each animation frame.
- **Blend.** Linear interpolation between mode A and mode B outputs: $\text{col} = (1 - \alpha) \text{col}_A + \alpha \text{col}_B$ in sRGB.

Colour-Vision Deficiency Simulation

CVD simulation is applied as a post-process in *linear* sRGB, after the pixel colour has been computed by any mode. The simulation matrix M_{cvd} multiplies the linear (R_ℓ, G_ℓ, B_ℓ) triplet:

$$\begin{pmatrix} R' \\ G' \\ B' \end{pmatrix}_\ell = M_{\text{cvd}} \begin{pmatrix} R_\ell \\ G_\ell \\ B_\ell \end{pmatrix} \quad (8)$$

The result is gamma-compressed back to sRGB. The matrices (Viénot et al. 1999, Brettel et al. 1997, simplified for real-time use) are:

$$M_{\text{deutan}} = \begin{pmatrix} 0.625 & 0.375 & 0 \\ 0.700 & 0.300 & 0 \\ 0 & 0.300 & 0.700 \end{pmatrix} \quad (\text{Deuteranopia — green receptor loss, } \sim 6\% \text{ of males})$$

$$M_{\text{protan}} = \begin{pmatrix} 0.567 & 0.433 & 0 \\ 0.558 & 0.442 & 0 \\ 0 & 0.242 & 0.758 \end{pmatrix} \quad (\text{Protanopia — red receptor loss, } \sim 2\% \text{ of males})$$

$$M_{\text{tritan}} = \begin{pmatrix} 0.950 & 0.050 & 0 \\ 0 & 0.433 & 0.567 \\ 0 & 0.475 & 0.525 \end{pmatrix} \quad (\text{Tritanopia — blue receptor loss, } \sim 0.01\%)$$

$$M_{\text{achrom}} = \begin{pmatrix} 0.299 & 0.587 & 0.114 \\ 0.299 & 0.587 & 0.114 \\ 0.299 & 0.587 & 0.114 \end{pmatrix} \quad (\text{Achromatopsia — full greyscale})$$

The pipeline order is: **pixel function** → **physics overlay** → **CVD simulation** → **canvas write**.

Rendering Pipeline

Sphere Projection

The sphere canvas is rendered pixel-by-pixel. For a canvas of width W and height H , a pixel at screen coordinates (p_x, p_y) maps to a surface normal as follows. Let $R = W/2 - 4$ be the sphere radius in pixels and $(c_x, c_y) = (W/2, H/2)$ the centre. Then:

$$s_x = (p_x - c_x)/R, \quad s_y = -(p_y - c_y)/R \quad (9)$$

$$s_z = \sqrt{\max(0, 1 - s_x^2 - s_y^2)} \quad (10)$$

Pixels with $s_x^2 + s_y^2 > R^2$ (outside the sphere disc) are left transparent.

Rotation

The viewer rotates the sphere by dragging, accumulating azimuthal and polar rotation angles (α, β) . The rotation matrix is:

$$M_{\text{rot}} = R_x(\beta) R_y(\alpha) \quad (11)$$

The surface normal seen at screen position (s_x, s_y, s_z) in world space is:

$$\hat{\mathbf{n}} = M_{\text{rot}} \begin{pmatrix} s_x \\ s_y \\ s_z \end{pmatrix} \quad (12)$$

Equirectangular Projection

The 2-D projection renders all positions $(\phi, n_z) \in [-\pi, \pi] \times [-1, 1]$ where ϕ is the azimuthal longitude. For a pixel at (p_x, p_y) in a $W \times H$ canvas:

$$n_z = 1 - 2p_y/(H - 1) \quad (13)$$

$$\phi = -\pi + 2\pi p_x/(W - 1) \quad (14)$$

$$n_x = \cos \phi \cdot \sqrt{1 - n_z^2}, \quad n_y = \sin \phi \cdot \sqrt{1 - n_z^2} \quad (15)$$

This projection is rotation-independent and reveals seams or discontinuities that might be hidden on the rotatable sphere.

Specular Glint

After writing the pixel image data, a radial gradient is composited over the sphere to simulate an overhead specular highlight:

```
gradient centre: (cx - 0.30 R, cy - 0.35 R) -- offset upper-left
gradient radius: 0.04 R --> R
colour stops:    rgba(255,255,255, 0.18) at 0
                  rgba(255,255,255, 0.00) at 0.5
```

Pattern Modes (Artefact 2)

All pattern modes accept a *palette* argument: an array of RGB triplets. The palette is decoupled from the mode and can be customised independently. The pixel function signature is:

$$\text{pixel} : (\hat{n}, \text{params}, \text{palette}) \mapsto [R, G, B]$$

Voronoi-Type Patterns

Octant (8 regions)

Partition S^2 into 8 octants by the sign of each component:

$$\text{index} = 4 \cdot [n_x \geq 0] + 2 \cdot [n_y \geq 0] + [n_z \geq 0] \quad (16)$$

Output: `palette[index]`.

Voronoi 6

Assign each point to its nearest axis pole under dot-product distance:

$$i^* = \arg \max_{i \in \{1, \dots, 6\}} \hat{\mathbf{n}} \cdot \hat{\mathbf{p}}_i \quad (17)$$

Output: `palette[i*]`.

Hemispheres

Select the pole with the largest absolute component, then use the sign:

$$\text{dominant axis} = \arg \max \{|n_x|, |n_y|, |n_z|\} \quad (18)$$

$$\text{colour} = \begin{cases} \text{palette}[0, 1] & \text{if dominant axis is } x, \text{ by sign} \\ \text{palette}[2, 3] & \text{if dominant axis is } y, \text{ by sign} \\ \text{palette}[4, 5] & \text{if dominant axis is } z, \text{ by sign} \end{cases} \quad (19)$$

Icosahedral

The 12 vertices of a regular icosahedron, normalised to S^2 , define 12 Voronoi cells. The icosahedron vertices are:

$$V = \{(0, \pm 1, \pm \varphi), (\pm 1, \pm \varphi, 0), (\pm \varphi, 0, \pm 1)\} / \|(0, 1, \varphi)\|$$

where $\varphi = (1 + \sqrt{5})/2$ is the golden ratio. Assign $i^* = \arg \max_i \hat{\mathbf{n}} \cdot \hat{\mathbf{v}}_i$ and output `palette[i* mod |palette|]`.

Fibonacci Lattice

The Fibonacci (golden-angle) spiral lattice generates N near-uniformly distributed points on S^2 . For $i = 0, \dots, N - 1$:

$$n_{z,i} = 1 - 2i/(N - 1) \quad (20)$$

$$r_i = \sqrt{\max(0, 1 - n_{z,i}^2)} \quad (21)$$

$$\phi_i = \pi(\sqrt{5} - 1) \cdot i \quad (\text{golden angle accumulation}) \quad (22)$$

$$\hat{\mathbf{p}}_i = (r_i \cos \phi_i, r_i \sin \phi_i, n_{z,i}) \quad (23)$$

Assign to nearest point: $i^* = \arg \max_i \hat{\mathbf{n}} \cdot \hat{\mathbf{p}}_i$. This distributes cells more uniformly than the icosahedron and avoids polar clustering.

Soft Voronoi

Rather than hard nearest-pole assignment, blend between the two nearest axis poles using a logistic (sigmoid) function:

$$t = \sigma(k_s \cdot (d_1 - d_2)), \quad \sigma(x) = \frac{1}{1 + e^{-x}} \quad (24)$$

where $d_1 \geq d_2$ are the top two dot products with axis poles. Output: $t \cdot \text{pal}[i_1] + (1 - t) \cdot \text{pal}[i_2]$.

Stripe Patterns

Checkerboard

Convert $\hat{\mathbf{n}}$ to spherical coordinates (θ, ϕ) with $\theta \in [0, \pi]$ and $\phi \in [0, 2\pi)$:

$$\theta = \arccos(n_z), \quad \phi = \text{atan2}(n_y, n_x) + \pi \quad (25)$$

The checkerboard index is:

$$\text{even} = \left(\left\lfloor \frac{f\theta}{\pi} \right\rfloor + \left\lfloor \frac{f\phi}{2\pi} \right\rfloor \right) \bmod 2 \quad (26)$$

For *integer* frequency f , the tiles close perfectly at $\phi = 2\pi$ (the antimeridian), giving a seamless checkerboard.

Latitude Stripes

$$\text{colour} = \begin{cases} \text{pal}[0] & \cos(f \arccos(n_z)) > 0 \\ \text{pal}[1] & \text{otherwise} \end{cases} \quad (27)$$

No atan2 involved; no seam possible.

Longitude Stripes

$$\text{colour} = \begin{cases} \text{pal}[0] & \sin(f \text{atan2}(n_y, n_x)) > 0 \\ \text{pal}[1] & \text{otherwise} \end{cases} \quad (28)$$

Seamless because $\sin(f \cdot \pi) = 0$ exactly for integer f : the seam falls precisely on a zero crossing of the function.

Truchet Mosaic

Divide S^2 into a $f \times f$ spherical patch grid using (θ, ϕ) . Each patch cell (c_i, c_j) receives a deterministic orientation bit:

$$h(c_i, c_j) = (\sin(127.1 c_i + 311.7 c_j) \cdot 43758.5) \bmod 1 \quad (29)$$

Within each patch, local coordinates $(u, v) \in [0, 1]^2$ determine the colour by a diagonal split:

$$\text{colour} = \begin{cases} \text{pal}[0] & (h > 0.5) \text{ and } (u + v < 1), \text{ or } (h \leq 0.5) \text{ and } (u < v) \\ \text{pal}[1] & \text{otherwise} \end{cases} \quad (30)$$

Overlay Modes

Grid Overlay

Compute VMF OKLAB base colour, then replace with `gridCol` if within a fractional distance $\epsilon = 0.04$ of any grid line:

$$\text{on grid} \iff (f_\theta \bmod 1 < \epsilon) \text{ or } (f_\phi \bmod 1 < \epsilon) \quad (31)$$

where $f_\theta = f\theta/\pi$ and $f_\phi = f\phi/(2\pi)$.

Iso-Hue Contours

Compute the VMF hue angle $\psi = \text{atan2}(b_{\text{vmf}}, a_{\text{vmf}}) \in [0, 2\pi)$. Replace pixels near iso-hue lines with `gridCol`:

$$\text{on contour} \iff ((N_c \cdot \psi / 2\pi) \bmod 1) < \epsilon \quad (32)$$

Gradient Magnitude

Approximate the colour-space gradient of the VMF map by finite differences with step $\varepsilon = 0.02$:

$$|\nabla \mathbf{c}| \approx \frac{1}{2} \sqrt{\|\mathbf{c}(\hat{\mathbf{n}} + \varepsilon \hat{x}) - \mathbf{c}(\hat{\mathbf{n}})\|^2 + \|\mathbf{c}(\hat{\mathbf{n}} + \varepsilon \hat{y}) - \mathbf{c}(\hat{\mathbf{n}})\|^2} \quad (33)$$

where $\mathbf{c}(\hat{\mathbf{n}})$ is the re-normalised surface point. Colours are normalised and mapped through a two-stop palette (basin interior $\rightarrow$ boundary). High gradient regions correspond to basin boundaries; on a fractal attractor these would trace out the fractal boundary structure.

3-D Perlin Noise

4-octave 3-D value noise evaluated at $(\hat{\mathbf{n}} \cdot s, \cdot s, \cdot s)$ where $s = 1.8f$:

$$v(\hat{\mathbf{n}}) = \sum_{k=0}^3 \frac{1}{2^k} \omega(2^k s n_x, 2^k s n_y, 2^k s n_z) \Bigg/ \sum_{k=0}^3 \frac{1}{2^k} \quad (34)$$

where ω is a smooth lattice interpolation (value noise). Evaluating in 3-D Cartesian coordinates instead of (θ, ϕ) guarantees no seam.

Special Modes

Real Spherical Harmonics

The real spherical harmonics $Y_\ell^m : S^2 \rightarrow \mathbb{R}$ for $\ell \in \{1, 2, 3\}$ and $|m| \leq \ell$ are:

$$\begin{aligned} Y_1^{-1} &= \sqrt{3/4\pi} n_y, & Y_1^0 &= \sqrt{3/4\pi} n_z, & Y_1^1 &= \sqrt{3/4\pi} n_x \\ Y_2^{-2} &= \sqrt{15/4\pi} n_x n_y, & Y_2^0 &= \sqrt{5/16\pi} (2n_z^2 - n_x^2 - n_y^2), & \dots \end{aligned}$$

(Higher ℓ terms follow standard Condon–Shortley phase conventions.) The normalised value $v = Y_\ell^m / \max |Y_\ell^m|$ is mapped to a colour by blending between a positive-lobe colour and a negative-lobe colour proportionally to $|v|$, with neutral grey at $v = 0$ (nodal lines).

Turing-Like Standing Waves

Three plane waves at 120° apart in the equatorial plane are superposed to produce a pattern with 3-fold symmetry (matching the 3-body problem’s C_3 symmetry):

$$v(n_x, n_y) = \frac{1}{3} \left[\sin(f \cdot n_x) + \sin(f(0.5n_x + 0.866n_y)) + \sin(f(0.5n_x - 0.866n_y)) \right] \quad (35)$$

The result $v \in [-1, 1]$ is linearly mapped to a two-colour palette.

Custom N-Pole VMF

Generalises the standard 6-pole scheme to N poles arranged on a tilted great circle. Pole directions are:

$$\hat{\mathbf{p}}_i = \begin{pmatrix} \cos(\phi_0 + 2\pi i/N) \cos \psi \\ \sin(\phi_0 + 2\pi i/N) \cos \psi \\ \sin \psi \end{pmatrix}, \quad i = 0, \dots, N-1 \quad (36)$$

where ϕ_0 is the equatorial rotation and $\psi = \text{tilt} \cdot \pi/2$ is the elevation of the pole ring above the equator. Hues are assigned as $\theta_i = 2\pi i/N$ (uniform spacing). The vMF blend of eq. (7) is used unchanged.

Stability $\times$ Hue

The core encoding for Principia. Hue encodes position via Okabe–Ito VMF; lightness encodes proximity to the binary collision singularities $\{\text{BC}_{12}, \text{BC}_{23}, \text{BC}_{31}\}$ on the shape sphere:

$$d_{\text{BC}} = \max_{j \in \{12, 23, 31\}} \hat{\mathbf{n}} \cdot \hat{\mathbf{b}}_j \quad (37)$$

$$\text{stability} = \frac{1}{2}(1 - d_{\text{BC}}) \in [0, 1] \quad (38)$$

$$L = 0.25 + 0.55 \cdot \text{stability} \quad (39)$$

where $d_{\text{BC}} \in [-1, 1]$ is the dot product with the nearest binary collision direction. Points near a collision ($d \approx 1$) are dark; points antipodal to all collisions are light.

Basin Blend

Soft interpolation between the two nearest Fibonacci Voronoi cells:

$$\text{col} = t \text{ pal}[i_1] + (1 - t) \text{ pal}[i_2], \quad t = \sigma(k_s (d_1 - d_2 - 0.04)) \quad (40)$$

The offset -0.04 shifts the transition slightly away from the exact boundary, giving a noticeable border region.

Dot Lattice

The Fibonacci lattice is rendered as dots rather than filled Voronoi cells. The angular radius of each dot scales with cell density:

$$\rho = \cos\left(\frac{1.4}{\sqrt{N}}\right) \quad (41)$$

A pixel is coloured if $\hat{\mathbf{n}} \cdot \hat{\mathbf{p}}_{i^*} > \rho$; otherwise it receives the background colour. This choice of ρ ensures dots cover approximately the same area regardless of N .

Shape Sphere Physics Overlay

Configuration Space

For the planar three-body problem with bodies of masses m_1, m_2, m_3 , the *shape sphere* (Montgomery’s representation) parametrises the relative configurations modulo overall scale and rotation. The unit sphere S^2 coordinatises the normalised shape of a triangle formed by the three bodies.

Three classes of special configurations are overlaid:

Symbol	Configuration	Location on S^2	Count
$\text{BC}_{12}, \text{BC}_{23}, \text{BC}_{31}$	Binary collisions	Equatorial, 120° apart	3
$\text{E}_1, \text{E}_2, \text{E}_3$	Euler collinear configs	Equatorial, between BCs	3
L^+, L^-	Lagrange equilateral	North and south poles	2

Explicitly, with the equal-mass conventions used here:

$$\hat{\mathbf{b}}_1 = (1, 0, 0), \quad \hat{\mathbf{b}}_2 = \left(-\frac{1}{2}, \frac{\sqrt{3}}{2}, 0\right), \quad \hat{\mathbf{b}}_3 = \left(-\frac{1}{2}, -\frac{\sqrt{3}}{2}, 0\right) \quad (42)$$

$$\hat{\mathbf{e}}_1 = (-1, 0, 0), \quad \hat{\mathbf{e}}_2 = \left(\frac{1}{2}, -\frac{\sqrt{3}}{2}, 0\right), \quad \hat{\mathbf{e}}_3 = \left(\frac{1}{2}, \frac{\sqrt{3}}{2}, 0\right) \quad (43)$$

$$\hat{\mathbf{l}}^\pm = (0, 0, \pm 1) \quad (44)$$

Gaussian Blob Overlay

Each special configuration blends its colour into the underlying pixel colour using a vMF blob:

$$\mathbf{c}_{\text{out}} = \mathbf{c}_{\text{base}} + \sum_j w_j (\mathbf{c}_j - \mathbf{c}_{\text{base}}) \quad (45)$$

where

$$w_j = s \cdot 4 \cdot \max(0, \exp(\kappa_j(\hat{\mathbf{n}} \cdot \hat{\mathbf{p}}_j - 1) + 0.01)) \quad (46)$$

and $s \in [0, 1]$ is the user-controlled strength, κ_j is the blob concentration (11 for BCs, 9 for Euler/Lagrange). The shifts are applied sequentially, which approximately implements additive blending for small w_j .

Palette System

Fixed- N Palettes

Modes requiring a fixed number of colours (e.g. $N = 2$ for binary patterns, $N = 6$ for Voronoi, $N = 8$ for octant) expose individual colour swatches. Each swatch binds to an HTML colour picker. Preset palettes include:

- **2-colour:** Okabe–Ito Blue/Orange, Okabe–Ito Teal/Purple, B&W, Indigo/Gold, Red/Cyan, Slate/Peach.
- **6-colour:** Okabe–Ito, Muted, Neon, Pastel, Principia.
- **8-colour:** Okabe–Ito, Earth, Neon.

Variable- N Palettes

Modes with a variable cell count N (Fibonacci, Dot lattice, Basin blend) use a generated palette. Three generation strategies are available:

1. **Golden angle.** Hue $\theta_i = 2\pi \cdot \{i \phi_g\}$ where $\phi_g = (\sqrt{5} - 1)/2$ is the fractional golden ratio. This spreads hues maximally uniformly: adjacent indices are ≈ 137.5 apart in hue, ensuring high contrast between neighbouring cells.
2. **Okabe–Ito cycle.** Colours sampled from the OI8 palette mod 8. CB-safe for all CVD types.
3. **Gradient.** Linear interpolation between two user-chosen endpoint colours: $\mathbf{c}_i = (1 - i/(N - 1)) \mathbf{c}_A + (i/(N - 1)) \mathbf{c}_B$ in sRGB.

Cubehelix Generation

The Cubehelix palette is generated analytically (not loaded from a LUT). For $t \in [0, 1]$:

$$\phi = 2\pi(s/3 - \lambda t), \quad a = h \cdot t(1 - t)/2 \quad (47)$$

$$R = t + a(-0.14861 \cos \phi + 1.78277 \sin \phi) \quad (48)$$

$$G = t + a(-0.29227 \cos \phi - 0.90649 \sin \phi) \quad (49)$$

$$B = t + a(1.97294 \cos \phi) \quad (50)$$

with start hue $s = 0.5$, number of rotations $\lambda = 1.5$, and saturation parameter $h = 1.0$.

Implementation Notes

Performance

All rendering is CPU-bound JavaScript operating on an `ImageData` buffer. The sphere canvas is 280×280 pixels; the equirectangular projection is 560×280 . Each full redraw iterates over all pixels

in the visible disc ($\approx \pi \cdot 136^2 \approx 58,000$ pixels) or the full projection (156,800 pixels). Expensive modes such as Fibonacci ($N = 96$) or Dot lattice perform $O(N)$ dot products per pixel; at $N = 96$ and 58k pixels this is $\approx 5.6 \times 10^6$ dot products per frame. This is acceptable on modern hardware but auto-rotate is recommended to be disabled for high- N modes.

Seam-Free Guarantee

A pattern is seam-free on S^2 if and only if its pixel function is continuous. The following modes guarantee this:

- **All vMF modes.** The vMF weight $\exp(\kappa \hat{\mathbf{n}} \cdot \hat{\mathbf{p}})$ is a smooth function of $\hat{\mathbf{n}}$; the weighted mean is a smooth composition.
- **Seamless LUT sphere.** Smooth by the same argument.
- **Latitude stripes.** $\cos(f \arccos(n_z))$ is smooth everywhere on S^2 .
- **Longitude stripes.** $\sin(f \operatorname{atan2}(n_y, n_x))$ has a potential discontinuity at the antimeridian, but the function value is continuous there for integer f (the seam falls on a zero crossing of $\sin$).
- **Perlin noise.** 3-D noise in Cartesian coordinates is smooth everywhere.

The following modes are *intentionally discontinuous* (hard boundaries):

- Octant, Voronoi 6, Hemispheres, Icosahedral, Fibonacci (hard), Checkerboard, Truchet.

Colour-Blindness Design Principles

The following design principles govern CB safety classification in this system:

1. A mode is **safe** if the deuteranopia and protanopia simulations preserve all discriminable pairs.
2. The critical failure mode is red–green confusion: modes using both red and green as distinct colours will fail under deuteranopia/protanopia.
3. The blue–yellow axis is preserved across all common CVD types, making it the most reliable encoding axis. The Okabe–Ito scheme is designed around blue–orange as the primary pair.
4. Sequential LUTs (Viridis, Cividis) encode position in lightness and are therefore partially readable even in full greyscale (achromatopsia), but only encode one dimension.
5. Cubehelix is CB-tolerant (not fully safe) because its monotone lightness means position can be approximately recovered from luminance alone, even when hue information is degraded.

Quick Reference

Parameters Summary

Parameter	Range	Effect
L	$[0.35, 0.90]$	OKLAB lightness; reserved for stability scalar
C	$[0.05, 0.22]$	OKLAB chroma; controls colour vibrancy
κ	$[0.5, 12]$	vMF concentration; controls blob spread
f	$[2, 14]$	Stripe/grid frequency (integer for seam-free)
N_c	$[4, 20]$	Number of contour lines
N	$[12, 96]$	Number of Fibonacci/dot cells
k_s	$[1, 20]$	Sigmoid sharpness for soft Voronoi/basin blend
ℓ, m	$\ell \in \{1, 2, 3\}, m \leq \ell$	Spherical harmonic degree and order
ψ	$[0, \pi/2]$	Custom VMF pole ring tilt
ϕ_0	$[0, 2\pi)$	Custom VMF pole ring rotation
s	$[0, 1]$	Physics overlay blob strength

Mode – CB Safety Summary

Mode	2-D encoding	Perceptually uniform	CB safety
VMF OKLAB	Full	✓	Deutan issues
VMF Okabe–Ito	Full	✓	Safe (all types)
Viridis/Cividis	Full (LUT)	✓	Safe / Optimised
Plasma/Magma/Inferno	Full (LUT)	✓	Partial
Twilight	Full (LUT, cyclic)	✓	Partial
Cubehelix	Full (LUT)	Monotone L	Tolerant
Dir. cosines	Full	×	Better greyscale
Voronoi/Octant/etc.	Full (discrete)	×	Depends on palette
Fibonacci	Full (discrete)	×	Safe (golden/OI pal.)
Stability×hue	Full + physics	✓	Safe
Harmonics Y_ℓ^m	Partial	Semi	Safe